Analiza Algoritmilor
- Pregătire Examen -

2023

Cuprins

1 Materie **1**
- 1.1 Calcularea Complexității Algoritmilor . . . 1
  - 1.1.1 Sfaturi pregătire . . . 2
- 1.2 Calculabilitate . . . 2
- 1.3 Determinism . . . 3
- 1.4 Corectitudine . . . 3
  - 1.4.1 Sfaturi pregătire . . . 4

2 Întrebări **4**
- 2.1 Complexitate . . . 4
- 2.2 Calculabilitate . . . 6
- 2.3 Determinism . . . 6
- 2.4 Corectitudine . . . 8

3 Soluții **9**
- 3.1 Complexitate . . . 9
- 3.2 Calculabilitate . . . 9
- 3.3 Determinism . . . 10
- 3.4 Corectitudine . . . 12

4 Referințe **14**
- 4.1 Complexitate . . . 14
- 4.2 Calculabilitate . . . 14
- 4.3 Determinism . . . 14
- 4.4 Corectitudine . . . 14

1 Materie

Putem să structurăm materia studiată la AA în patru secțiuni importante:

1.1 Calcularea Complexității Algoritmilor

Noțiuni esențiale:

- Să recunoaștem notațiile asimptotice și să putem verifica proprietățile acestora.
- Să putem calcula complexitatea unei secvențe de cod iterativ/recursiv folosind oricare din metodele studiate.
  - Să identificăm relația de recurență care caracterizează o secțiune de cod.
  - Să putem aplica metoda iterației, metoda arborelui de recurență sau teorema Master pentru o recurență dată.
- Cum funcționează metoda creditelor/metoda potențialului (analiză amortizată)?

1.1.1 Sfaturi pregătire

- Nu trebuie să reproduceți complet demonstrația de la Teorema Master dar dacă înțelegeți pașii principali (*de ce* comparăm $n^{log_b(a)}$ cu $f(n)$) o să vă fie mult mai ușor să o rețineți.
- Asigurați-vă că se îndeplinesc constrângerile necesare pentru a putea aplica Teorema Master: a și b sunt **constante**, $a \geq 1$, $b > 1$, $f(n)$ asimptotic pozitivă.
- Nu uitați de acel ϵ din formulă! Pentru a aplica Teorema Master $f(n)$ aceasta trebuie să fie **polinomial** mai mică (cazul 1) sau mai mare (cazul 3) decât $n^{log_b(a)}$. De exemplu, expresia $\frac{n}{log(n)}$ **nu** este polinomial mai mică decât n.
- Reluați exemplele din seminar. Poate fi un exercițiu util să încercați să abordați o recurență prin mai multe metode.

1.2 Calculabilitate

Noțiuni esențiale:

- Ce înseamnă că o problemă este recursivă/nerecursivă/recursiv-enumerabilă?
- În ce situații practice ne putem întâlni cu o problemă nerecursivă?
  - În particular, mă interesează să rămâneți cu rezultatul Teoremei Rice: Nu putem verifica o proprietate semantică netrivială (extensională) a unui program.
  - Putem verifica proprietăți sintactice (dacă un program iterativ are 3 for-uri sau o mașină Turing are 10 reguli de tranziție) dar nu putem construi un solver general, care primește la intrare un program și spune cum va arăta ieșirea acestuia în funcție de intrare (sau dacă va cicla).
  - Implicit, **nu putem verifica automat corectitudinea unui program arbitrar** (dacă ieșirea programului are anumite proprietăți semantice).
- Nu putem explora un spațiu infinit de posibilități fără a avea o informație suplimentară care să ne permită să restrângem spațiul de căutare.

Noțiuni (mai) avansate:

- Cum verificăm că o reducere many-one ($A <_m B$) este corectă?
  - Funcția F de transformare este calculabilă.
  - $A(w) = 1 \rightarrow B(F(w)) = 1$.
  - $B(F(w)) = 1 \rightarrow A(w) = 1$.
- Cum putem demonstra că o problemă de decizie A este recursivă?
  - Construim programul dorit.
  - Arătăm că A se reduce (many-one sau Turing) la o problemă Q despre care știu că este recursivă ($A <_m Q$).
- Cum putem demonstra că o problemă de decizie A este recursiv enumerabilă (RE)?
  - Construim un program care să răspundă corect, în timp finit, pentru orice instanță a problemei, dacă răspunsul este DA; Programul poate să cicleze dacă răspunsul este NU.
  - Arătăm că putem reduce A la o problemă $Q \in RE$ ($A <_m Q$).
- Cum putem demonstra că o problema de decizie A este nerecursivă?
  - Putem demonstra prin reducere la absurd (v. Halting Problem, Busy Beaver).
  - Alternativ, putem folosi o reducere many-one/reducere Turing de la o problemă Q despre care știm că este nerecursivă ($Q <_m A$).

1.3 Determinism

Noțiuni esențiale:

- Ce reprezintă clasele de complexitate P, NP, NP-Hard și NP-Complete?
- Ce implică $P = NP$?

Noțiuni (mai) avansate:

- Cum verificăm că o reducere many-one polinomială ($A <_p B$) este corectă?
  - Funcția F de transformare poate fi implementată în timp polinomial determinist.
  - $A(w) = 1 \rightarrow B(F(w)) = 1$.
  - $B(F(w)) = 1 \rightarrow A(w) = 1$.
- Cum demonstrez că o problemă de decizie A este în P?
  - Construiesc un algoritm polinomial determinist.
  - Arăt că A se reduce polinomial la o altă problemă Q din P ($A <_p Q$).
- Cum demonstrez că o problemă de decizie A este în NP?
  - Construiesc un algoritm polinomial nedeterminist care rezolvă A.
  - Arăt că A se reduce polinomial la o altă problemă Q din NP ($A <_p Q$).
- Cum demonstrez că o problemă de decizie A este în NP-Hard?
  - Arăt că există o problemă Q din NP-Hard care se reduce la A. ($Q <_p A$).
  - Teorema Cook-Levin: Orice problemă din NP se reduce la problema SAT.
- Cum demonstrez că o problemă de decizie A este în NPC?
  - Arăt că există o problemă Q din NP-Hard care se reduce la A ($Q <_p A$) și $A \in NP$.
- Probleme specifice: SAT, K-Colorare, Subset Sum
- Ierarhia claselor de complexități: $L \subseteq NL \subseteq P \subseteq NP \subseteq PSPACE = NPSPACE \subseteq EXP$.

Tips & Tricks:

- Pentru a implementa un algoritm nedeterminist polinomial ambele etape (generarea certificatului, validarea sa) trebuie implementate în timp **polinomial**.
- Atenție! Operația **choice** primește la intrare **o mulțime finită** și întoarce un element din respectiva mulțime. Exemple valide: mulțimea nodurilor dintr-un graf, mulțimea codurilor ASCII, etc.
- Dacă vrem ca algoritmul să fie polinomial, dacă mulțimea nu aparține datelor de intrare, aceasta trebuie să poată fi construită în timp polinomial.

1.4 Corectitudine

Noțiuni Esențiale:

- Noțiunea de corectitudine parțială/totală.
- Noțiunea de invariant la ciclare:
  - Cum alegem un invariant (ce proprietăți vrem să testăm)?
  - Cum verificăm un invariant?
- Cum verificăm o proprietate a unei operații aplicate asupra unui tip de date T definit recursiv folosind inducție structurală?

1.4.1 Sfaturi pregătire

- Recapitulați exemplele studiate la seminar.
- Inducție structurală:
  - La examen puteți introduce notații noi pentru a scrie mai puțin (de ex.: "Notăm sorted() cu s()"). Menționați noile notații la începutul demonstrației.
  - Nu săriți pași! Dacă o proprietate necesară este '*evidentă*' probabil este necesară o proprietate adițională care trebuie demonstrată prin inducție.
  - De asemenea, o proprietate adițională este necesară atunci când nu mai putem înainta in demonstrație folosind doar axiomele/proprietățile cunoscute.
  - **Precizați clar axiomele folosite** pentru fiecare pas din demonstrație.
  - Dacă aveți de analizat rezultatul unei expresii condiționale (if), **tratați separat fiecare caz**.
  - $False \rightarrow Anything.$

2 Întrebări

2.1 Complexitate

1. Calculați complexitatea asimptotică pentru următoarea recurență: $T(n) = 3 * T(\frac{3*n}{10}) + n * \sqrt{n} - O(n), T(1) = 1.$
2. Fie funcțiile $f, g : \mathbb{N} \rightarrow R_+$, monoton crescătoare: $f(n) \in \Theta(log^2(n))$, $g(n) \in O(\sqrt{n})$. Determinați, cât mai strict, limite asimptotice pentru următoarele expresii: $f(n) + g(n)$ (5p) și $f(g(n))$ (5p).
3. Fie următoarea recurență:

$$T(n) = T(\frac{n}{2}) + T(\sqrt{n}) + n^2 \tag{1}$$

   - Găsiți limite inferioare și superioare cât mai exacte pentru complexitatea recurenței, comparând-o cu alte recurențe mai ușor de calculat.
   - Validați prin metoda substituției (inducție) că recurența se încadrează între limitele asimptotice găsite.

4. Fie Q, un algoritm adaptiv. Q începe prin a preprocesa datele de intrare folosind algoritmul A. Apoi, dacă dimensiunea datelor de intrare este suficient de mare ($n > k$, unde k este o constantă), apelează algoritmul B, altfel apelează algoritmul C.

$$A(n) = A(\frac{n}{4}) + c_1 \cdot log(n), A(1) = 1 \tag{2}$$

$$B(n) = B(\frac{n}{4}) + B(3 \cdot \frac{n}{4}) + c_2 \cdot n, B(1) = 1 \tag{3}$$

$$C(n) = C(n-1) + c_3 \cdot n^2, C(1) = 1. \tag{4}$$

   Determinați complexitatea asimptotică, folosind orice metodă, pentru algoritmii A (4p), B (4p), C (4p) respectiv Q (3p).

5. Verificați dacă se poate aplica teorema Master și calculați complexitatea următoarei recurențe:

$$T(n) = 3T(\frac{n}{2}) + n\sqrt{n}\log n \tag{5}$$

6. Calculați, prin orice metodă, complexitatea următoarei recurențe (Θ):

$$T(n) = T(\sqrt[3]{n}) + T(\sqrt[4]{n}) + \log(n) \tag{6}$$

7. Algoritmul M are asociată următoarea recurență:

$$T(n) = \begin{cases} T(\lceil \frac{n}{5} \rceil) + T(7 \cdot \frac{n}{10} + 6) + c \cdot n & n \geq 140 \\ O(1) & n < 140 \end{cases} \tag{7}$$

- Determinați o limită inferioară, respectiv superioară pentru varianta simplificată a recurenței: $T(n) = T(\frac{n}{5}) + T(7 \cdot \frac{n}{10}) + c * n$. Puteți folosi orice metodă studiată.
- Validați că recurența inițială se încadrează între aceleași limite asimptotice.

8. Având un vector de n elemente numere întregi ca intrare, vrem să calculăm valoarea **minimă** pentru fiecare secvență de k elemente din acest vector. Rezultatul final al algoritmului va fi **maximul** acestor valori minime calculate.

   Pentru a implementa eficient această funcție, vom folosi un **deque** despre care știm că are o complexitate amortizată de $O(1)$ pentru toate operațiile: *pushFront(x)*, *pushBack(x)*, *popFront()*, *popBack()*. De asemenea, structura oferă acces la primul și ultimul element prin câmpurile *first* și *last*.

```
1: function FINDMIN(deque, k, i)
2:     while len(deque) > 0 & v[i] ≤ v[deque.last] do
3:         deque.popBack()
4:     end while
5:     deque.pushBack(i)
6:     if i − deque.first ≥ k then
7:         deque.popFront()
8:     end if
9:     return deque.first
10: end function

11: function MAXMIN(v, n, k)
12:     deque = []
13:     max = −∞
14:     for (i = 0; i < n; i++) do
15:         min = v[FINDMIN(deque, k, i)]
16:         if i ≥ k − 1 & min > max then
17:             max = min
18:         end if
19:     end for
20:     return max
21: end function
```

   Demonstrați, prin orice metodă, că funcția *FINDMIN* are un cost amortizat constant atunci când este folosită în cadrul funcției *MAXMIN*.

9. Calculați complexitatea temporală a funcției f:

```
int f(int[] v) {
    n = v.size()
    if (n < 2) {
        return 0;
    }
    m = n / 2;
    s = f(v[0..m−1]) + f(v[m..n−1]);
    for (int i = 1; i < n; i *= 2) {
        for (int j = n−1; j >= i; j−−) {
            s += v[j] / v[i];
        }
    }
    return s;
}
```

2.2 Calculabilitate

1. Fie două probleme de decizie A și B. Știm că $A <_T B$ și $B \in RE$. Atunci:

   - ◯ $A \in RE$
   - ◯ A poate fi recursivă.
   - ◯ A poate fi nerecursivă.
   - ◯ A nu poate fi nerecursivă.

2. Fie Q o problemă asfel încât $\overline{Q} \in R$. Atunci:

   - ◯ $Q \in R$.
   - ◯ $Q \in RE$.
   - ◯ $Q \notin R$.
   - ◯ $Q \notin RE$.

3. Fie următoarea problemă:

$$f(P,k) = \begin{cases} 1 & \text{, P se oprește pe cel puțin K intrări} \\ 0 & \text{, altfel} \end{cases}$$

   Care afirmații sunt adevărate?

   - ◯ $f \in R$
   - ◯ $f \in RE$
   - ◯ $HALT \leq_T f$
   - ◯ $HALT \leq_T \overline{f}$

4. Fie f o funcție recursiv enumerabilă, dar nu recursivă. Care dintre următoarele funcții sunt recursiv enumerabile?

   - ◯ g(x) = {1, dacă $f(x) = 1$, 0 altfel}
   - ◯ g(x) = {1, dacă $f(x) = 1$ & $f(x+1) = 0$, 0 altfel}
   - ◯ g(x) = {1, dacă $f(x) = 1$ & $f(2x) = 1$, 0 altfel}
   - ◯ g(x) = {1, dacă $f(x) = 0$, 0 altfel}

2.3 Determinism

1. Fie f, g, h - 3 probleme astfel încât $f \leq_P g$ și $g \leq_P h$. Dacă $f \in NPC$ și $g \in NP$, atunci:

   - ◯ $g \in NPC$
   - ◯ $h \in NPC$
   - ◯ $g \in$ NP-Hard
   - ◯ $h \in$ NP-Hard

2. Presupunând că P=NP, care dintre următoarele afirmații sunt adevărate?

   - ◯ Toate problemele din NP pot fi rezolvate în timp polinomial cu un algoritm determinist.
   - ◯ Toate problemele din NP-Hard pot fi rezolvate în timp polinomial cu un algoritm determinist.
   - ◯ NPC=NP-Hard

3. Să presupunem că problema X este în clasa NP, problema Y în clasa NPC și $X \leq_p Y$. Care din următoarele afirmații sunt sigur adevărate?

   - ◯ $X \notin P$
   - ◯ $X \in P$
   - ◯ $X \in NPC$.

- ◯ $Y \in NP$

4. Presupunând că ați găsit o reducere polinomială de la 3-COLOR la 4-COLOR. Care variante sunt sigur adevărate?

   - ◯ 4-COLOR $\in NP$
   - ◯ 4-COLOR $\in$ NP-Hard
   - ◯ 3-COLOR $\in P$
   - ◯ $P = NP$

5. Fie următoarea reducere polinomială de la *3-clique* la *4-clique*: Construim graful G' adăugând la graful G (intrarea pentru *3-clique*) 3 noi noduri conectate între ele și cu toate nodurile din G. **Observație:** *k-clique* verifică dacă există un subgraf complet de dimensiune k într-un graf. Care afirmație este adevărată?

   - ◯ Reducerea polinomială este corectă.
   - ◯ 3-clique$(G) = 1 \iff$ 4-clique$(G') = 1$, dar reducerea nu este polinomială.
   - ◯ Doar implicația 3-clique$(G) = 1 \Rightarrow$ 4-clique$(G') = 1$ este adevărată
   - ◯ Doar implicația 4-clique$(G') = 1 \Rightarrow$ 3-clique$(G) = 1$ este adevărată

6. Fie următoarea reducere polinomială de la *partition* la *3-partition*. Construim mulțimea $V' = V \cup \{S/2\}$ ca intrare pentru *3-partition*, unde V este intrarea pentru *partition*, iar $S = \sum_{x \in V} x$.

   Observație: *Partition* verifică dacă o mulțime de numere întregi poate fi împărțită în două submulțimi disjuncte de sumă egală, în timp ce *3-partition* verifică dacă o mulțime poate fi împărțită în 3 submulțimi disjuncte de sumă egală.

   Care afirmație este adevărată?

   - ◯ Reducerea polinomială este corectă.
   - ◯ partition$(V) = 1 \iff$ 3-partition$(V') = 1$, dar reducerea nu este polinomială.
   - ◯ Doar implicația partition$(V) = 1 \Rightarrow$ 3-partition$(V') = 1$ este adevărată
   - ◯ Doar implicația 3-partition$(V') = 1 \Rightarrow$ partition$(V) = 1$ este adevărată

7. Prof. John a găsit o reducere polinomială de la SAT la problema opririi (PO) și de la SAT la o problemă Q din NP. Prin urmare:

   - ◯ $Q <_p SAT$
   - ◯ $PO <_p SAT$
   - ◯ $Q <_p PO$
   - ◯ $PO <_p Q$

8. Cum am putea demonstra că o problemă de decizie Q este în P? [1]

   - ◯ Elaborăm un algoritm nedeterminist polinomial care rezolvă problema Q.
   - ◯ Demonstrăm $Q <_p GAP$.
   - ◯ Demonstrăm $GAP <_p Q$.
   - ◯ Demonstrăm $Q <_p$ 2-Colorare.

9. Presupunem că $P \neq NP$. Care dintre următoarele afirmații sunt adevărate:

   - ◯ Problemele din NP-Hard nu pot fi rezolvate de un algoritm determinist în timp polinomial.
   - ◯ Problemele din NP-Hard pot fi rezolvate de un algoritm determinist în timp exponențial.
   - ◯ Un algoritm determinist care rezolvă o problemă din NP-Hard funcționează pentru orice intrare în timp $\omega(n^k), \forall k \in \mathbb{R}$.
   - ◯ Un algoritm determinist care rezolvă o problemă din NP-Hard funcționează pentru orice intrare în timp exponențial.

[1] GAP(G(V, E), S, D): Există un drum elementar de la nodul S la nodul D în graful G?

10. SubsetSum(A, T): Fie o mulțime de numere întregi A și o valoare țintă T. Putem alege un subset S din A atfel încât suma elementelor din S să fie egală cu T? Dacă presupunem că $P \neq NP$, care dintre următoarele afirmații sunt adevărate:

    - ◯ Problema SubsetSum poate fi rezolvată în timp polinomial determinist dacă toate numerele din A sunt mai mici sau egale cu n^3, unde n reprezintă numărul de elemente din A.
    - ◯ Problema SubsetSum poate fi rezolvată în timp polinomial determinist dacă numărul de elemente din A este mai mic decât T.
    - ◯ Problema SubsetSum nu poate fi rezolvată de un algoritm determinist în timp polinomial.
    - ◯ Problemele din NP-Complete nu pot fi rezolvate de un algoritm determinist, care este întotdeauna corect, în timp polinomial.

11. $Q \in DTIME(t(n)) \rightarrow Q \in NTIME(?)$ (upper-bound)
12. $Q \in NTIME(t(n)) \rightarrow Q \in DTIME(?)$ (upper-bound)
13. $Q \in DTIME(t(n)) \rightarrow Q \in DSPACE(?)$ (upper-bound)
14. Fie Q o problemă recursivă, $t(n) \geq n, Q \in DSPACE(t(n)) \rightarrow Q \in DTIME(?)$. (upper-bound)
15. Pentru $t(n) \geq n$, $Q \in NSPACE(t(n)) \rightarrow Q \in DSPACE(?)$ (upper-bound)

2.4 Corectitudine

List 1: Constructori

- $[] :\rightarrow ListT$
- $(x : xs) : T \rightarrow ListT \rightarrow ListT$

List 2: $member : T \rightarrow ListT \rightarrow Bool$

M1: $member(e, []) = False$

M2: $member(e, x : xs) = (x == e) \vee member(e, xs)$

List 3: $unique : ListT \rightarrow Bool$

U1: $unique([]) = True$

U2: $unique(x : xs) = !member(x, xs) \wedge unique(xs)$

List 4: $intersection : ListT \rightarrow ListT \rightarrow ListT$

INT1: $intersection([], y) = []$

INT2: $intersection(x : xs, y)$
$= x : intersection(xs, y)$, **if** $member(x, y)$

INT3: $intersection(x : xs, y)$
$= intersection(xs, y)$, **if** $!member(x, y)$

De demonstrat P1

$unique(l1) \wedge unique(l2) \implies unique(intersection(l1, l2))$.

3 Soluții

3.1 Complexitate

1. Calculați complexitatea asimptotică pentru următoarea recurență: $T(n) = 3 * T(\frac{3 \cdot n}{10}) + n * \sqrt{n} - O(n), T(1) = 1$.

   Barem generic:

   - Identificare corectă: $a = 3, b = \frac{10}{3}$
   - Identificare corectă: $f(n) = n * \sqrt{n} - O(n) = O(n^{1.5})$
   - Stabilit că: $n^{log_b(a)} = O(n^{1.5-\epsilon})$ cu $\epsilon > 0$ (depinde cum este făcută discuția, cel mai simplu ar fi să arate că $3 < \frac{10}{3}^{\frac{3}{2}}$)
   - Discutat + demonstrat condiția de regularizare pentru $f(n)$.
   - Concluzionat că putem aplica cazul 3 din Teorema Master și trasă concluzia ca $T(n) = \theta(n * \sqrt{n}) = \theta(n^{1.5})$

3.2 Calculabilitate

1. Fie două probleme de decizie A și B. Știm că $A <_T B$ și $B \in RE$. Atunci:

   - ◉ $A \in RE$
   - ◉ A poate fi recursivă.
   - ◉ A poate fi nerecursivă.
   - ○ A nu poate fi nerecursivă.

2. Fie Q o problemă asfel încât $\overline{Q} \in R$. Atunci:

   - ◉ $Q \in R$.
   - ◉ $Q \in RE$.
   - ○ $Q \notin R$.
   - ○ $Q \notin RE$.

3. Fie următoarea problemă:

   $$f(P, k) = \begin{cases} 1 & \text{, P se oprește pe cel puțin K intrări} \\ 0 & \text{, altfel} \end{cases}$$

   Care afirmații sunt adevărate?

   - ○ $f \in R$
   - ◉ $f \in RE$
   - ◉ $HALT \leq_T f$
   - ◉ $HALT \leq_T \overline{f}$

4. Fie f o funcție recursiv enumerabilă, dar nu recursivă. Care dintre următoarele funcții sunt recursiv enumerabile?

   - ◉ g(x) = {1, dacă $f(x) = 1$, 0 altfel}
   - ○ g(x) = {1, dacă $f(x) = 1$ & $f(x + 1) = 0$, 0 altfel}
   - ◉ g(x) = {1, dacă $f(x) = 1$ & $f(2x) = 1$, 0 altfel}
   - ○ g(x) = {1, dacă $f(x) = 0$, 0 altfel}

5. Fie problemele de decizie:

   Hamilton(G(V,E)) - verifică dacă există în graful G un drum care să treacă prin toate nodurile grafului o singură dată (drum hamiltonian).

   TSP(G(V, E), W, s) - verifică dacă există un drum hamiltonian cu costul $\leq s$ în graful G, unde funcția $W : E \to \mathbb{R}$ asociază un cost fiecărei muchii.

   a) (4p) Demonstrați că $TSP \in NP$ construind un algoritm nedeterminist cu complexitate polinomială.
   b) (4p) Demonstrați că $TSP \in$ NP-Hard arătând că $Hamilton \leq_P TSP$.

3.3 Determinism

1. Fie f, g, h - 3 probleme astfel încât $f \leq_P g$ și $g \leq_P h$. Dacă $f \in NPC$ și $g \in NP$, atunci:

   - ◉ $g \in NPC$
   - ○ $h \in NPC$
   - ◉ $g \in$ NP-Hard
   - ◉ $h \in$ NP-Hard

2. Presupunând că P=NP, care dintre următoarele afirmații sunt adevărate?

   - ◉ Toate problemele din NP pot fi rezolvate în timp polinomial cu un algoritm determinist.
   - ○ Toate problemele din NP-Hard pot fi rezolvate în timp polinomial cu un algoritm determinist.
   - ○ NPC=NP-Hard

3. Să presupunem că problema X este în clasa NP, problema Y în clasa NPC și $X \leq_p Y$. Care din următoarele afirmații sunt sigur adevărate?

   - ○ $X \notin P$
   - ○ $X \in P$
   - ○ $X \in NPC$.
   - ◉ $Y \in NP$

4. Presupunând că ați găsit o reducere polinomială de la 3-COLOR la 4-COLOR. Care variante sunt sigur adevărate?

   - ◉ 4-COLOR $\in NP$
   - ◉ 4-COLOR $\in$ NP-Hard
   - ○ 3-COLOR $\in P$
   - ○ $P = NP$

5. Fie următoarea reducere polinomială de la *3-clique* la *4-clique*: Construim graful G' adăugând la graful G (intrarea pentru *3-clique*) 3 noi noduri conectate între ele și cu toate nodurile din G. **Observație:** *k-clique* verifică dacă există un subgraf complet de dimensiune k într-un graf. Care afirmație este adevărată?

   - ○ Reducerea polinomială este corectă.
   - ○ 3-clique$(G) = 1 \iff$ 4-clique$(G') = 1$, dar reducerea nu este polinomială.
   - ◉ Doar implicația 3-clique$(G) = 1 \Rightarrow$ 4-clique$(G') = 1$ este adevărată
   - ○ Doar implicația 4-clique$(G') = 1 \Rightarrow$ 3-clique$(G) = 1$ este adevărată

6. Fie următoarea reducere polinomială de la *partition* la *3-partition*. Construim mulțimea $V' = V \cup \{S/2\}$ ca intrare pentru *3-partition*, unde V este intrarea pentru *partition*, iar $S = \sum_{x \in V} x$.

   Observație: *Partition* verifică dacă o mulțime de numere întregi poate fi împărțită în două submulțimi disjuncte de sumă egală, în timp ce *3-partition* verifică dacă o mulțime poate fi împărțită în 3 submulțimi disjuncte de sumă egală.

   Care afirmație este adevărată?

   - ◉ Reducerea polinomială este corectă.
   - ○ partition$(V) = 1 \iff$ 3-partition$(V') = 1$, dar reducerea nu este polinomială.
   - ○ Doar implicația partition$(V) = 1 \Rightarrow$ 3-partition$(V') = 1$ este adevărată
   - ○ Doar implicația 3-partition$(V') = 1 \Rightarrow$ partition$(V) = 1$ este adevărată

7. Prof. John a găsit o reducere polinomială de la SAT la problema opririi (PO) și de la SAT la o problemă Q din NP. Prin urmare:

   - ◉ $Q <_p SAT$

- ○ $PO <_p SAT$
- ◉ $Q <_p PO$
- ○ $PO <_p Q$

8. Cum am putea demonstra că o problemă de decizie Q este în P? [2]

   - ○ Elaborăm un algoritm nedeterminist polinomial care rezolvă problema Q.
   - ◉ Demonstrăm $Q <_p GAP$.
   - ○ Demonstrăm $GAP <_p Q$.
   - ◉ Demonstrăm $Q <_p$ 2-Colorare.

9. **Presupunem că $P \neq NP$.** Care dintre următoarele afirmații sunt adevărate:

   - ◉ Problemele din NP-Hard nu pot fi rezolvate de un algoritm determinist în timp polinomial.
   - ○ Problemele din NP-Hard pot fi rezolvate de un algoritm determinist în timp exponențial[3].
   - ○ Un algoritm determinist care rezolvă o problemă din NP-Hard funcționează pentru orice intrare în timp $\omega(n^k), \forall k \in \mathbb{R}$[4].
   - ○ Un algoritm determinist care rezolvă o problemă din NP-Hard funcționează pentru orice intrare în timp exponențial.

10. SubsetSum(A, T): Fie o mulțime de numere întregi A și o valoare țintă T. Putem alege un subset S din A atfel încât suma elementelor din S să fie egală cu T? **Dacă presupunem că $P \neq NP$**, care dintre următoarele afirmații sunt adevărate:

    - ◉ Problema SubsetSum poate fi rezolvată în timp polinomial determinist dacă toate numerele din A sunt mai mici sau egale cu n^3, unde n reprezintă numărul de elemente din A[5].
    - ○ Problema SubsetSum poate fi rezolvată în timp polinomial determinist dacă numărul de elemente din A este mai mic decât T.
    - ◉ Problema SubsetSum nu poate fi rezolvată de un algoritm determinist în timp polinomial[6].
    - ◉ Problemele din NP-Complete nu pot fi rezolvate de un algoritm determinist, care este întotdeauna corect, în timp polinomial[7].

11. $Q \in TIME(t(n)) \rightarrow Q \in SPACE(t(n))$

    - O mașină Turing care se termină în t(n) pași nu poate accesa mai mult de t(n) celule.

12. Fie Q o problemă recursivă, $t(n) \geq n, Q \in SPACE(t(n)) \rightarrow Q \in TIME(2^{O(t(n))})$.

    - Care este numărul maxim posibil de configurații prin care poate trece o mașina Turing care accesează t(n) celule fără a cicla?
    - O configurație este identificată prin: $Q \times H \times T$ (starea MT, poziția capului de bandă, valorile scrise efectiv pe bandă pe cele t(n) celule accesibile).
    - Numărul de stări este independent de datele de intrare.
    - $H \leq t(n)$.
    - Avem $2^{O(t(n))}$ posibilități de a completa banda.

13. $Q \in DTIME(t(n)) \rightarrow Q \in NTIME(t(n))$

    - Ne propunem să stabilim o limită superioară.
    - Un algoritm determinist poate fi simulat direct pe un model de calcul nedeterminist.

[2] GAP(G(V, E), S, D): Există un drum elementar de la nodul S la nodul D în graful G?
[3] Contra-exemplu: $HALT \in NP - Hard$
[4] Hint: Pentru *orice* intrare.
[5] Am studiat la curs un algoritm cu o complexitate temporală $O(N * W)$, unde W este valoarea maximă pe care o putem obține cu elementele din A.
[6] Deoarece am presupus că $P \neq NP$ și $SubsetSum \in NPC$.
[7] Altfel, toate problemele din NP ar fi în P, ceea ce ar contrazice presupunerea din întrebare.

14. $Q \in NTIME(t(n)) \rightarrow Q \in DTIME(2^{O(t(n))})$

    - Ne propunem să stabilim o limită superioară.
    - Fie b, numărul maxim de tranziții posibile la fiecare pas. Atunci, întregul arbore de execuție are cel mult $b^{t(n)}$ noduri. Astfel, putem simula determinist toate deciziile luate de algoritm în timp $2^{O(t(n))}$.

15. Pentru $t(n) \geq n$, $Q \in NSPACE(t(n)) \rightarrow Q \in DSPACE(t(n)^2)$

    - Teorema lui Savitch
    - Putem determina dacă există o cale între două noduri dintr-un graf G(V, E) folosind spațiu adițional $O(log^2(|V|))$.
    - Graful orientat determinat de configurațiile unei mașini Turing nedeterministe care accesează cel mult $t(n)$ celule are cel mult $|V| = 2^{O(t(n))}$ noduri. Avem muchie între două noduri A și B dacă este posibilă o tranziție între cele două configurații asociate nodurilor, conform specificației MT.
    - Fără să construim explicit acest graf în memorie, putem verifica dacă există o cale de la configurația inițială la o configurație care corespunde unei stări de acceptare a intrării folosind $log^2(|V|) = O(t(n)^2)$ spațiu adițional.
    - Același rezultat poate fi obținut pentru $t(n) = log(n)$.

3.4 Corectitudine

Barem generic

- **20%** Demonstrația pentru cazul de bază.
- **20%** Specificarea ipotezei inductive și a pasului de inducție.
- **30%** Demonstrația pasului de inducție.
- **10%** Identificarea proprietății adiționale (dacă este necesară).
- **20%** Demonstrația proprietății adiționale (dacă este necesară).

Rezolvare

- **Cazul de bază**: $l1 == []$

  Ne propunem să demonstrăm:
  $unique([]) \wedge unique(l2) \implies unique(intersection([], l2))$.
  $unique(intersection([], l2)) =^{INT1} unique([]) =^{M1} True$.

- **Ipoteza inductivă**: P(xs)
  $unique(xs) \wedge unique(l2) \implies unique(intersection(xs, l2))$.

- **Pasul de inducție**: $P(xs) \implies P(x : xs)$.

  Ne propunem să demonstrăm:
  $unique(x : xs) \wedge unique(l2) \implies unique(intersection(x : xs, l2))$.

  Presupunem $unique(x : xs)$, respectiv $unique(l2)$ adevărate, în caz contrar, implicația este trivial adevărată. Cum $unique(x : xs)$ este adevărat, putem folosi din ipoteza inductivă faptul că $unique(intersection(xs, l2))$ este adevărat.

  (O1) $unique(x : xs) \implies^{U2} !member(x, xs) \wedge unique(xs)$
  Fie $E = unique(intersection(x : xs, l2))$.

  Avem de analizat două cazuri:

1. $member(x, l2)$
   $E =^{INT2} unique(x : intersection(xs, l2))$.

   $E =^{U2} \ !member(x, intersection(xs, l2)) \wedge unique(intersection(xs, l2))$.

   Pentru a demonstra că expresia E este adevărată avem nevoie de o proprietate adițională PA(xs):
   $!member(e, xs) \implies !member(e, intersection(xs, L)), \forall L : ListT$

   $E =^{O1,PA,II} True$
2. $!member(x, l2)$
   $E =^{INT3} unique(intersection(xs, l2)) =^{II} True$.

Mai avem de demonstrat proprietatea adițională PA(xs):
$!member(e, xs) \implies !member(e, intersection(xs, L)), \forall L : ListT$.

- **Cazul de bază** $xs == []$.

  Ne propunem să demonstrăm:
  $!member(e, []) \implies !member(e, intersection([], L)), \forall L : ListT$.
  $E =!member(x, intersection([], L)) =^{(INT1)} !member(x, []) =^{(M1)} True$.

- **Ipoteza inductivă** $P(xs)$.
  $!member(e, xs) \implies !member(e, intersection(xs, L)), \forall L : ListT$

- **Pasul de inducție** $P(xs) \implies P(x : xs)$.

  Ne propunem să demonstrăm că:
  $!member(e, x : xs) \implies !member(e, intersection(x : xs, L)), \forall L : ListT$.

  Presupunem $!member(e, x : xs) = True$.
  (O1) $!member(e, x : xs) = (e \neq x) \wedge \ !member(e, xs)$.
  Fie $E =!member(e, intersection(x : xs, L))$

  Avem de analizat două cazuri:

  1. $member(x, L)$
     $E =^{INT2} !member(e, x : intersection(xs, L))$
     $E =^{M2} (e \neq x) \wedge !member(e, intersection(xs, L)) =^{O1,II} True$.

  2. $!member(x, L)$ $E =^{INT3} \ !member(e, intersection(xs, L)) =^{II} True$.

4 Referințe

4.1 Complexitate

Resursa de referință pentru prima parte a cursului (analiza complexității) este Introduction to Algorithms [3] (folosită ca suport principal la MIT). Este cartea care tratează în întregime și cel mai riguros subiectele pe care le-am atins si noi: notațiile asimptotice, metoda iterației, metoda arborilor de recurență, metoda substituției, Teorema Master și analiza amortizată.

În comparație, Algorithms Illuminated [4] (folosita la Stanford/Harvard) are o prezentare mai prietenoasă a materiei. Chiar dacă câteva dintre subiectele pe care ne concentrăm noi lipsesc (este mai potrivită pentru Proiectarea Algoritmilor de pe semestrul 2), oferă un background solid pentru cineva care vrea sa înțeleagă cum se calculează complexitatea în practică, cu multe aplicații concrete. Una din cărțile mele favorite.

O alternativă faină este și Algorithms [5] (Princeton) care militează pentru calcularea mai granulară a complexității. Mai exact, evită să încadreze expresia finală folosind notațiile asimptotice pentru a putea diferenția, de exemplu, între n^2 și $n^2 + nlogn$. De asemeni, pentru unii algoritmi studiați, analizează separat numărul de operații și în funcție de tipul acestora (ex: nr. operații de swap vs nr. operații de comparare).

4.2 Calculabilitate

În limba română există cartea prof. Giumale [1] care este însă mai greu de accesat. Referința mea preferată este cartea Introduction to the Theory of Computation [6] (v. cursul asociat de la MIT, PDF + Video Lectures).

4.3 Determinism

Un clip scurt care introduce bine subiectul poate fi accesat aici.
Pentru o întelegere bună recomand [4] (friendly) sau [3] (more verbose). Cea mai completă referință rămâne [6].

4.4 Corectitudine

Referințele principale sunt [2] (Capitolul 1) (v. curs MIT) și cartea prof. Giumale [1].

References

[1] Cristian A Giumale. *Introducere ın analiza algoritmilor: teorie şi aplicaţie*. Polirom, 2004.

[2] Eric Lehman. "Mathematics for computer science". In: (2004).

[3] Charles E Leiserson et al. *Introduction to algorithms*. MIT press, 2009.

[4] Tim Roughgarden. *Algorithms illuminated*. Soundlikeyourself publishing, 2017.

[5] Robert Sedgewick and Kevin Wayne. *Algorithms: Part I*. Addison-Wesley Professional, 2014.

[6] Michael Sipser. *Introduction to the Theory of Computation*. English. 3rd ed. Cengage Learning, 2012, p. 504. ISBN: 978-1133187790.